

O que é PDO?

O PDO (PHP Data Objects) é uma camada de abstração para acesso a banco de dados no PHP. Ele permite conectar e manipular diferentes bancos utilizando praticamente a mesma estrutura de código.

Com PDO você mysql, SQLite, PostgreSQL, SQL Server, Oracle entre outros, a principal vantagem é que precisamos apenas alterar o tipo de conexão.

Antigamente era necessário utilizar funções específicas para cada tipo de banco de dados.

exemplo:

```
mysql_connect(); // MySQL antigo
```

```
pg_connect(); // PostgreSQL
```

```
sqlite_open(); // SQLite
```

Porém isso trazia alguns problemas de segurança, código preso ao banco, dificuldade de manutenção, assim o PDO surgiu para padronizar e tornar mais fácil a utilização do banco de dados.

Como utilizar o PDO:

O PDO tem 3 partes principais, que são a conexão, a preparação da query e a execução, tendo assim um fluxo básico:

PHP -> PDO -> Driver do banco -> Banco de Dados

Para realizar a conexão com o banco precisamos instanciar um objeto do tipo PDO, passando uma string "mysql:host=localhost;dbname=teste", isso se chama DSN (Data Source Name). Assim definimos o tipo de banco "mysql", o host que é o caminho para nosso banco "localhost" e o nome do banco, também podemos passar o charset e a porta ele utilizará a padrão do driver no caso do mysql a 3306.

Também podemos passar nessa mesma string o usuário e senha de acesso ao banco, ou passar como parâmetro separado. Aqui temos um exemplo simples, passando o valor do usuário "root" e a senha em branco.

```
3
4 $pdo = new PDO(
5     "mysql:host=localhost;dbname=teste",
6     "root",
7     ""
8 );
9
```

Caso utilize o SQLite o DSN ficara dessa forma:

```
2
3 $pdo = new PDO("sqlite:banco.db");
4
```

Lembrando que o sqlite é um arquivo que você tem que colocar o caminho de onde o arquivo está salvo.

Executando o PDO:

Existe 3 formas principais de executar o PDO, `exec()` que é utilizado para INSERT, UPDATE e DELETE simples, `query()` que é utilizado para SELECT simples e o `prepare()` para queries mais seguras.

Exemplo com `exec()`, nesse exemplo estamos adicionando um usuário ao banco, você pode concatenar a string com variáveis para tornar dinâmico o algoritmo.

```
1 <?php
2
3 $pdo = new PDO("sqlite:banco.db");
4
5 $sql = "INSERT INTO usuarios(nome) VALUES('Leonardo')";
6
7 $pdo->exec($sql);
```

Problema do `exec` é que ele é inseguro, podendo ter um SQL injection como um `$nome = "";` `DROP TABLE usuarios; --`; que faria o drop da tabela. Porém para aprendizado podemos utilizar dessa forma sem problemas, aprenderemos a utilizar o `prepare` mais para frente.

Exemplo com query():

Nesse exemplo estamos buscando todos os dados do usuario no banco de dados.

```
index.php > ...
1  <?php
2
3  $pdo = new PDO("sqlite:banco.db");
4
5  $sql = "SELECT * FROM usuarios";
6
7  $stmt = $pdo->query($sql);
8
9  $usuarios = $stmt->fetchAll();
10
11 print_r($usuarios);
```

Entendendo linha por linha, primeiramente vamos criar uma instância do PDO, após isso criamos a string de seleção e adicionamos na variável \$sql (Lembrando que podemos usar o nome que quiser para variáveis), após isso executamos ela dentro do de uma outra variável que é o \$stmt para que ele aplique no \$stmt a execução da query. Por final utilizamos a função do pdo que é o fetchAll() para recebermos todos os dados do banco através de um array.

Também podemos utilizar o fetch quando é apenas um registro do banco de dados.